

数字电路设计

——DDCA 第 4 章硬件描述语言 HDL 详细学习笔记

基于 *Digital Design and Computer Architecture: ARM[®] Edition*

Sarah L. Harris & David Money Harris

2026 年 6 月 15 日

目录

1. 引言：HDL 概述与设计流程 (p.1–p.9)
2. SystemVerilog 模块基础 (p.10–p.16)
3. 组合逻辑的 HDL 描述 (p.18–p.35)
4. 结构建模 (Structural Modeling) (p.17, p.25)
5. 时序逻辑的 HDL 描述 (p.36–p.42)
6. 更多组合逻辑：always_comb、case/casez、阻塞与非阻塞赋值 (p.43–p.49)
7. 有限状态机 (FSM) 的 HDL 实现 (p.50–p.52)
8. 参数化模块 (Parameterized Modules) (p.53)

9. 测试平台 (Testbench) (p.54–p.66)
10. 考点速查与复习指南

1 引言：HDL 概述与设计流程

1.1 什么是硬件描述语言（HDL）？

[p.3]

HDL (Hardware Description Language): 仅指定逻辑功能，CAD 工具（计算机辅助设计）产生或综合出优化的门级电路网表。大多数商业设计都使用 HDL 构建。

两大主流 HDL: [p.3]

- **SystemVerilog / Verilog**
 - 1984 年由 Gateway Design Automation 开发
 - 1995 年成为 IEEE 标准（IEEE STD 1364）
 - 2005 年扩展为 SystemVerilog（IEEE STD 1800-2009）
- **VHDL 2008**
 - 1981 年由美国国防部开发
 - 1987 年成为 IEEE 标准（IEEE STD 1076）
 - 2008 年更新（IEEE STD 1076-2008）

1.2 HDL 到门电路的设计流程

[p.4–p.5]

仿真 (Simulation): [p.4]

- 将输入施加到电路上
- 检查输出是否正确
- 在仿真中调试可以节省数百万美元（相比直接在硬件上调试）

综合 (Synthesis): [p.4]

- 将 HDL 代码转换为描述硬件的网表（netlist）
- 网表是门电路及其连接导线的列表

重要提醒 [p.5]: 使用 HDL 时，始终思考 HDL 应该产生什么样的硬件！不能把 HDL 当软件程序来写。

1.3 Digital 工具与 Verilog

[p.7–p.9]

- Digital 工具可通过菜单 文件 → 导出 → Verilog 自动生成 Verilog 代码 [p.7]
- iVerilog: 免费的 Verilog 仿真器和 SystemVerilog 支持 [p.8]
- GTKWave: 查看仿真波形的工具 [p.8]
- 在 Digital 中配置 iVerilog: 菜单 Edit → Settings [p.9]

2 SystemVerilog 模块基础

2.1 模块 (Module) 概念

[p.10]

两种模块类型 [p.10]:

1. 行为建模 (**Behavioral**): 描述模块做什么 (what it does)
2. 结构建模 (**Structural**): 描述模块如何由更简单的模块构建 (how it is built from simpler modules) —— 层次化设计

2.2 行为建模示例

[p.11, p.13]

```
module example(input logic a, b, c,  
               output logic y);  
    assign y = ~a & ~b & ~c | a & ~b & ~c | a & ~b & c;  
endmodule
```

关键语法说明 [p.13]:

- module / endmodule: 必需的开始/结束关键字
- example: 模块名称
- 操作符: ~ = NOT, & = AND, | = OR

注意: 也可以使用”与-或”混合方式来设计, 但 **Digital** 不能配置 **GTKWave** [p.6]。

2.3 SystemVerilog 语法规则

[p.16]

- 大小写敏感: reset 和 Reset 不是同一个信号
- 名称不能以数字开头: 2mux 是无效名称
- 空白符被忽略
- 注释:
 - 单行注释: //
 - 多行注释: /* ... */

2.4 仿真与综合

[p.14–p.15]

- **HDL 仿真** [p.14]: 用 iVerilog 等工具对模块进行功能验证
- **HDL 综合** [p.15]: 用 Vivado 等工具将 HDL 代码综合为门级网表
- 本书推荐使用 **Digital + HDL 混合方法**: 用 Digital 画原理图, 导出 Verilog, 或用 iVerilog 仿真 [p.6–p.9]

3 组合逻辑的 HDL 描述

3.1 按位操作符 (Bitwise Operators)

[p.18]

```
module gates(input logic [3:0] a, b,
             output logic [3:0] y1, y2, y3, y4, y5);
    assign y1 = a & b;           // AND
    assign y2 = a | b;           // OR
    assign y3 = a ^ b;           // XOR
    assign y4 = ~(a & b);        // NAND
    assign y5 = ~(a | b);        // NOR
endmodule
```

3.2 缩减操作符 (Reduction Operators)

[p.19]

```
module and8(input logic [7:0] a,
            output logic y);
    assign y = &a;
    // &a 比 assign y = a[7]&a[6]&a[5]&a[4]&a[3]&a[2]&a[1]&a[0] 简洁
endmodule
```

缩减操作符将多 bit 信号缩减为 1bit: $\&a$ (AND 缩减)、 $|a$ (OR 缩减)、 $\^a$ (XOR 缩减) 等。

3.3 条件赋值 (Conditional Assignment)

[p.20]

```
module mux2(input logic [3:0] d0, d1,
            input logic s,
            output logic [3:0] y);
    assign y = s ? d1 : d0;    // 三元操作符 (ternary operator)
endmodule
```

?: 是三元操作符, 操作在 3 个输入上: s 、 $d1$ 、 $d0$ 。当 $s=1$ 时选 $d1$, 当 $s=0$ 时选 $d0$ 。

3.4 内部变量 (Internal Variables)

[p.21]

```
module fulladder(input logic a, b, cin,
                 output logic s, cout);
    logic p, g;           // 内部节点 (internal nodes)
    assign p = a ^ b;
    assign g = a & b;
    assign s = p ^ cin;
    assign cout = g | (p & cin);
endmodule
```

使用 logic 类型声明内部信号。

3.5 操作符优先级

[p.22]

优先级	操作符	说明
最高	~	NOT
	*, /, %	乘、除、取模
	+, -	加、减
	«, »	逻辑移位
	«<, »>	算术移位
	<, <=, >, >=	比较
	==, !=	相等、不等
	&, ~&	AND, NAND
	^, ~^	XOR, XNOR
	, ~	OR, NOR
最低	?:	三元操作符

3.6 数字常量 (Numbers)

[p.23]

格式: N'Bvalue

- N = 位数 (bit 数), B = 进制基数 (base)
- N'B 是可选的但建议使用 (默认为十进制)

数字	#Bits	进制	十进制	存储值
3'b101	3	binary	5	101
'b11	unsized	binary	3	00...0011
8'b11	8	binary	3	00000011
8'b1010_1011	8	binary	171	10101011
3'd6	3	decimal	6	110
6'o42	6	octal	34	100010
8'hAB	8	hex	171	10101011
42	unsized	decimal	42	00...0101010

注意: 下划线 _ 仅用于格式化, SystemVerilog 会忽略它们。

3.7 位操作 (Bit Manipulations)

[p.24–p.25]

拼接操作符 {}: [p.24]

```
assign y = {a[2:1], {3{b[0]}}, a[0], 6'b100_010};
// 结果: y = a[2] a[1] b[0] b[0] b[0] a[0] 1 0 0 0 1 0
```

复制操作符 {N{signal}} 将信号复制 N 次。

3.8 三态输出 (Z: Floating Output)

[p.26]

```
module tristate(input logic [3:0] a,
               input logic en,
               output tri [3:0] y);
    assign y = en ? a : 4'bz;    // z 表示高阻态
endmodule
```

z 值表示高阻 (floating) 输出, 用于三态缓冲器。输出类型需声明为 tri。

3.9 延迟 (Delays)

[p.27–p.35]

```
module example(input logic a, b, c,
               output logic y);
    logic ab, bb, cb, n1, n2, n3;
```

```
assign #1 {ab, bb, cb} = ~{a, b, c};  
assign #2 n1 = ab & bb & cb;  
assign #2 n2 = a & bb & cb;  
assign #2 n3 = a & bb & c;  
assign #4 y = n1 | n2 | n3;  
endmodule
```

- #N 指定信号延迟 N 个时间单位
- `timescale 1ns/1ps 在 testbench 中设定时间单位 [p.28]
- 延迟主要用于仿真，综合时被忽略

用 iVerilog 仿真 [p.28–p.29]: 编写 testbench (tb_example.sv), 用 \$dumpfile 和 \$dumpvars 导出 VCD 波形文件, 用 GTKWave 查看波形。

用 VSCode 编辑 SystemVerilog [p.30]: 推荐使用 VSCode 配合 SystemVerilog 插件进行代码编辑。

4 结构建模 (Structural Modeling)

4.1 层次化设计

[p.17]

```
module and3(input logic a, b, c,
            output logic y);
    assign y = a & b & c;           // 行为建模
endmodule

module inv(input logic a,
            output logic y);
    assign y = ~a;                 // 行为建模
endmodule

module nand3(input logic a, b, c,
              output logic y);
    logic n1;                      // 内部信号
    and3 andgate(a, b, c, n1);     // and3的实例
    inv inverter(n1, y);           // inv的实例
endmodule                          // 结构建模
```

关键点:

- 结构建模通过实例化 (instantiate) 已有模块来构建更复杂的模块
- 内部信号用 `logic` 声明
- 实例化语法: 模块名实例名 (端口连接列表);

4.2 多 bit 实例化

[p.25]

```
// 用两个4-bit mux构建8-bit mux
module mux2_8(input logic [7:0] d0, d1,
              input logic s,
              output logic [7:0] y);
    mux2 lsbmux(d0[3:0], d1[3:0], s, y[3:0]);
    mux2 msbmux(d0[7:4], d1[7:4], s, y[7:4]);
endmodule
```

5 时序逻辑的 HDL 描述

5.1 SystemVerilog 时序习语 (Idioms)

[p.36]

SystemVerilog 使用特定的习语 (idioms) 来描述锁存器 (latches)、触发器 (flip-flops) 和有限状态机 (FSMs)。其他编码风格可能仿真正确但产生错误的硬件!

5.2 Always 语句

[p.37]

```
always @(sensitivity list)
    statement;
```

每当敏感列表中的事件发生时, 语句就执行。

5.3 D 触发器 (D Flip-Flop)

[p.38]

```
module flop(input logic clk,
            input logic [3:0] d,
            output logic [3:0] q);
    always_ff @(posedge clk) // 时钟上升沿触发
        q <= d;             // 非阻塞赋值 <=
endmodule
```

always_ff 明确表示意图是描述触发器,<= 是非阻塞赋值 (nonblocking assignment)。

5.4 带同步复位的 D 触发器

[p.39]

```
module flopr(input logic clk,
             input logic reset,
             input logic [3:0] d,
             output logic [3:0] q);
    // synchronous reset (同步复位)
    always_ff @(posedge clk)
```

```
        if (reset) q <= 4'b0;
        else      q <= d;
    endmodule
```

同步复位仅在时钟上升沿检查 reset 信号。

5.5 带异步复位的 D 触发器

[p.40]

```
module flopr(input logic clk,
             input logic reset,
             input logic [3:0] d,
             output logic [3:0] q);
    // asynchronous reset (异步复位)
    always_ff @(posedge clk, posedge reset)
        if (reset) q <= 4'b0;
        else      q <= d;
endmodule
```

关键区别: 异步复位将 reset 也放入敏感列表中, 这样 reset 上升沿也能触发 always 块。

5.6 带使能的 D 触发器

[p.41]

```
module flopren(input logic clk,
               input logic reset,
               input logic en,
               input logic [3:0] d,
               output logic [3:0] q);
    // enable and asynchronous reset
    always_ff @(posedge clk, posedge reset)
        if (reset)      q <= 4'b0;
        else if (en)    q <= d;
endmodule
```

en 信号控制是否更新输出。注意 en 和 reset 信号位置。

5.7 锁存器 (Latch)

[p.42]

```
module latch(input logic clk,  
             input logic [3:0] d,  
             output logic [3:0] q);  
    always_latch  
        if (clk) q <= d;          // 电平敏感（非边沿触发）  
endmodule
```

警告 [p.42]: 本书不使用锁存器。但是你可能无意中写出隐含锁存器的代码。检查综合后的硬件——如果里面有锁存器，说明代码有错误。

6 更多组合逻辑：always_comb、case/casez、阻塞与非阻塞赋值

6.1 Always 中的其他行为语句

[p.43]

- if / else —— 条件判断
- case、casez —— 多路分支
- 这些语句必须放在 always 块内部

6.2 用 always_comb 描述组合逻辑

[p.44]

```
module gates(input logic [3:0] a, b,
             output logic [3:0] y1, y2, y3, y4, y5);
    always_comb // 用always_comb描述组合逻辑
    begin      // 多条语句需要begin/end
        y1 = a & b; // 注意：阻塞赋值 =
        y2 = a | b;
        y3 = a ^ b;
        y4 = ~(a & b);
        y5 = ~(a | b);
    end
endmodule
```

提示: 这种硬件用 assign 语句行数更少, 因此这种情况下使用 assign 更好。只有在复杂组合逻辑时才使用 always_comb。

6.3 用 case 描述组合逻辑

[p.45–p.46]

```
module sevenseg(input logic [3:0] data,
                output logic [6:0] segments);
    always_comb
    case (data)
        //                abc_defg
```

```

0: segments = 7'b111_1110;
1: segments = 7'b011_0000;
2: segments = 7'b110_1101;
3: segments = 7'b111_1001;
4: segments = 7'b011_0011;
5: segments = 7'b101_1011;
6: segments = 7'b101_1111;
7: segments = 7'b111_0000;
8: segments = 7'b111_1111;
9: segments = 7'b111_0011;
default: segments = 7'b000_0000; // 必须的default!
endcase
endmodule

```

case 语句关键规则 [p.46]:

- case 语句仅当所有可能的输入组合都有描述时才意味着组合逻辑
- 务必使用 default 语句，否则可能产生锁存器（latch）

6.4 用 casez 描述优先级逻辑

[p.47]

```

module priority_casez(input logic [3:0] a,
                      output logic [3:0] y);

always_comb
  casez(a)
    4'b1??? : y = 4'b1000; // ? = don't care
    4'b01?? : y = 4'b0100;
    4'b001? : y = 4'b0010;
    4'b0001 : y = 4'b0001;
    default : y = 4'b0000;
  endcase
endmodule

```

casez 中 ? 表示不关心（don't care），用于实现优先级编码器。

6.5 阻塞赋值 vs 非阻塞赋值

[p.48]

两种赋值方式:

- `<=` : 非阻塞赋值 (Nonblocking) ——同时发生 (并行)
- `=` : 阻塞赋值 (Blocking) ——按文件中出现顺序执行 (顺序)

对比示例——同步器 (Synchronizer) : [p.48]

Good (非阻塞) :

```
module syncgood(  
    input logic clk, d,  
    output logic q);  
    logic n1;  
    always_ff @(posedge clk)  
        begin  
            n1 <= d; // nonblocking  
            q <= n1; // nonblocking  
        end  
endmodule
```

Bad (阻塞) :

```
module syncbad(  
    input logic clk, d,  
    output logic q);  
    logic n1;  
    always_ff @(posedge clk)  
        begin  
            n1 = d; // blocking  
            q = n1; // blocking  
        end  
endmodule
```

6.6 信号赋值规则总结

[p.49]

信号赋值四规则

1. 同步时序逻辑: 使用 `always_ff @(posedge clk)` 和非阻塞赋值 (`<=`)
2. 简单组合逻辑: 使用连续赋值 (`assign ...`)
3. 复杂组合逻辑: 使用 `always_comb` 和阻塞赋值 (`=`)
4. 一个信号只能在一个 `always` 语句或一个 `assign` 语句中赋值

7 有限状态机 (FSM) 的 HDL 实现

7.1 FSM 的三模块结构

[p.50]

FSM 由三个模块组成:

1. 次态逻辑 (Next State Logic) —— 组合逻辑
2. 状态寄存器 (State Register) —— 时序逻辑
3. 输出逻辑 (Output Logic) —— 组合逻辑

7.2 FSM 实例: 除以 3 (Divide by 3)

[p.51–p.52]

状态转换图: 三个状态 S0, S1, S2 循环转换 (S0→S1→S2→S0), 双圆圈表示复位状态 (S0)。[p.51]

```
module divideby3FSM(input logic clk,
                    input logic reset,
                    output logic q);
    typedef enum logic [1:0] {S0, S1, S2} statetype;
    statetype [1:0] state, nextstate;

    // 状态寄存器 (state register) — 时序逻辑
    always_ff @(posedge clk, posedge reset)
        if (reset) state <= S0;
        else      state <= nextstate;

    // 次态逻辑 (next state logic) — 组合逻辑
    always_comb
        case (state)
            S0: nextstate = S1;
            S1: nextstate = S2;
            S2: nextstate = S0;
            default: nextstate = S0;
        endcase
```

```
// 输出逻辑 (output logic) — 组合逻辑
assign q = (state == S0);
endmodule
```

FSM 编码要点:

- 使用 `typedef enum logic [N:0]` 定义状态编码 [p.52]
- 状态寄存器用 `always_ff + <=` (非阻塞赋值)
- 次态逻辑用 `always_comb + =` (阻塞赋值)
- 输出逻辑可用 `assign` 或 `always_comb`
- 异步复位: 敏感列表含 `posedge reset`

8 参数化模块 (Parameterized Modules)

8.1 参数化 2:1 多路复用器

[p.53]

```
module mux2
  #(parameter width = 8)      // 参数名和默认值
  (input logic [width-1:0] d0, d1,
   input logic s,
   output logic [width-1:0] y);
  assign y = s ? d1 : d0;
endmodule
```

实例化方式:

- 使用默认值 (8-bit) :

```
mux2 myMux(d0, d1, s, out);
```

- 指定参数值 (12-bit) :

```
mux2 #(12) lowmux(d0, d1, s, out);
```

`#(parameter name = default_value)` 语法在模块名后、端口列表前定义参数。

9 测试平台 (Testbench)

9.1 Testbench 概述

[p.54]

Testbench: 测试另一个模块的 HDL 代码。被测试模块称为 DUT (Device Under Test) 或 UUT (Unit Under Test)。

Testbench 不可综合 (not synthesizable), 仅用于仿真。

三种类型 [p.54]:

1. **Simple testbench** ——简单手动赋值
2. **Self-checking testbench** ——自动比较结果
3. **Self-checking with testvectors** ——从文件读取测试向量

9.2 待测模块: sillyfunction

[p.55–p.56]

实现功能: $y = \bar{b} \bar{c} + a \bar{b}$

```
module sillyfunction(input logic a, b, c,
                    output logic y);
    assign y = ~b & ~c | a & ~b;
endmodule
```

9.3 简单 Testbench

[p.57]

```
module testbench1();
    logic a, b, c;
    logic y;
    sillyfunction dut(a, b, c, y);    // 实例化DUT
    initial begin
        a = 0; b = 0; c = 0; #10;
        c = 1; #10;
        b = 1; c = 0; #10;
        c = 1; #10;
        a = 1; b = 0; c = 0; #10;
```

```
c = 1; #10;
b = 1; c = 0; #10;
c = 1; #10;
end
endmodule
```

9.4 自检测 Testbench

[p.58]

使用 \$display 和 if (y !== expected) 进行自动比较:

```
module testbench2();
    logic a, b, c;
    logic y;
    sillyfunction dut(a, b, c, y);
    initial begin
        a = 0; b = 0; c = 0; #10;
        if (y !== 1) $display("000 failed.");
        c = 1; #10;
        if (y !== 0) $display("001 failed.");
        b = 1; c = 0; #10;
        if (y !== 0) $display("010 failed.");
        // ... (覆盖全部8种输入组合)
    end
endmodule
```

!== 和 === 可以比较包含 x 和 z 的值。

9.5 带测试向量的 Testbench

[p.59–p.66]

测试向量文件 (example.tv) [p.61]:

```
000_1
001_0
010_0
011_0
100_1
101_1
110_0
```

111_0

格式: abc_yexpected (输入 __ 预期输出)

完整 Testbench 实现 (4 个部分):

生成时钟 [p.62]:

```
module testbench3();
    logic clk, reset;
    logic a, b, c, yexpected;
    logic y;
    logic [31:0] vectornum, errors;
    logic [3:0] testvectors[10000:0];    // 测试向量数组

    sillyfunction dut(a, b, c, y);      // 实例化DUT

    always          // 无敏感列表 → 始终执行
    begin
        clk = 1; #5; clk = 0; #5;      // 周期10ns
    end
```

读取测试向量 [p.63]:

```
initial
begin
    $readmemb("example.tv", testvectors); // 读取二进制文件
    vectornum = 0; errors = 0;
    reset = 1; #27; reset = 0;            // 复位脉冲
end
// 注: $readmemb 读取十六进制测试向量文件
```

赋值输入和预期输出 [p.64]:

```
always @(posedge clk)    // 时钟上升沿赋值
begin
    #1; {a, b, c, yexpected} = testvectors[vectornum];
end
```

比较输出与预期 [p.65–p.66]:

```
always @(negedge clk)    // 时钟下降沿比较
    if (~reset) begin    // 跳过复位期间
```

```
    if (y !== yexpected) begin
        $display("Error: inputs = %b", {a, b, c});
        $display("  outputs = %b (%b expected)", y, yexpected);
        errors = errors + 1;
    end
    vectornum = vectornum + 1;
    if (testvectors[vectornum] === 4'bx) begin // 检测向量末尾
        $display("%d tests completed with %d errors",
                vectornum, errors);
        $finish;
    end
end
endmodule
```

时钟时序 [p.60]:

- 上升沿 (posedge): 赋值输入
- 下降沿 (negedge): 比较输出与预期

注意: testbench 的时钟也可用作同步时序电路的时钟 [p.60]。

10 考点速查与复习指南

10.1 核心概念对照表

概念	关键字/语法	用途
模块定义	module / endmodule	基本设计单元 [p.13]
连续赋值	assign y = ...	简单组合逻辑 [p.49]
组合逻辑 always 块	always_comb	复杂组合逻辑 [p.44]
时序逻辑 always 块	always_ff @(posedge clk)	触发器、FSM [p.38]
锁存器 always 块	always_latch	锁存器（不推荐）[p.42]
阻塞赋值	=	组合逻辑内部 [p.48]
非阻塞赋值	<=	时序逻辑 [p.48–p.49]
case/casez	case (signal) ... endcase	多路选择/优先级 [p.45–p.47]
三元操作符	?:	条件赋值/MUX [p.20]
位拼接	{}	位操作 [p.24]
高阻态	z, tri	三态输出 [p.26]
参数化	#(parameter name = val)	可重用模块 [p.53]
测试向量读取	\$readmemb / \$readmemh	Testbench [p.63]
状态枚举	typedef enum logic	FSM 状态编码 [p.52]

10.2 高频考点

1. 阻塞 vs 非阻塞赋值的区别 [p.48–p.49]

- 非阻塞 <=: 并行执行，用于时序逻辑（always_ff）
- 阻塞 =: 顺序执行，用于组合逻辑（always_comb）
- 错误示例：在 always_ff 中使用阻塞赋值会导致同步器失效

2. 同步复位 vs 异步复位 [p.39–p.40]

- 同步: always_ff @(posedge clk), reset 仅在时钟沿检测
- 异步: always_ff @(posedge clk, posedge reset), reset 上升沿立即触发

3. case 语句的 default [p.46]

- 缺少 default 会导致隐含锁存器（latch）
- 综合工具会推断出非预期的硬件

4. FSM 三模块结构 [p.50–p.52]

- 次态逻辑 (`always_comb, =`)
- 状态寄存器 (`always_ff, <=`)
- 输出逻辑 (`assign` 或 `always_comb`)
- 推荐使用 `typedef enum` 定义状态

5. Testbench 不可综合 [p.54]

- `initial`、`$display`、`$finish` 等仅用于仿真
- `$readmemb` 读取二进制测试向量
- `$readmemh` 读取十六进制测试向量

6. 参数化模块实例化 [p.53]

- 默认参数: `mux2 myMux(d0, d1, s, out);`
- 覆盖参数: `mux2 #(12) lowmux(d0, d1, s, out);`

10.3 常见错误提醒

- 在 `always_ff` 中使用阻塞赋值 —— 导致错误的硬件行为 [p.48]
- `case` 缺少 `default` —— 产生意外锁存器 [p.46]
- 信号在多个 `always` 块中赋值 —— 多驱动错误 [p.49]
- 不完整的敏感列表 —— 仿真与综合结果不一致 [p.37]
- 认为 HDL 是软件程序 —— 必须始终想着硬件结构 [p.5]
- 变量名以数字开头 —— 语法错误 [p.16]

10.4 仿真与综合工具链

工具	功能	说明
Digital	画原理图 + 导出 Verilog	免费开源 [p.7]
iVerilog	Verilog/SystemVerilog 仿真	免费 [p.8]
GTKWave	查看 VCD 波形	免费 [p.8]
Vivado	HDL 综合	Xilinx/AMD [p.15]
VSCode	代码编辑	配合 SV 插件 [p.30]

——END OF CHAPTER 4 NOTES ——

本文档基于 Digital Design and Computer Architecture: ARM[®] Edition (Harris & Harris,
2015) 第 4 章整理

生成日期: 2026 年 6 月 15 日